

Laboratório 03

AUTHOR

Julia Maria Pereira Pinheiro

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.
- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;
- Para o arquivo de vôos:
 - a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;
 - e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
airlines <- read_csv( "dados/airlines.csv")
```

Rows: 14 Columns: 2

— Column specification —

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

! Use ``spec()`` to retrieve the full column specification for this data.

! Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
airports <- read_csv(
  "dados/airports.csv",
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)
```

Rows: 322 Columns: 5

— Column specification —

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

! Use ``spec()`` to retrieve the full column specification for this data.

! Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
getStats <- function(input, pos) {

  input %>%
    filter(!grepl("^1", DESTINATION_AIRPORT)) %>%
    filter(
      !is.na(DESTINATION_AIRPORT),
```

```

    !is.na(ARRIVAL_DELAY)
  ) %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma = sum(ARRIVAL_DELAY),
    total = n(),
    .groups = "drop"
  )
}

colunas <- cols_only(
  DESTINATION_AIRPORT = col_character(),
  ARRIVAL_DELAY = col_double()
)
flights <- read_csv_chunked(
  file = "dados/flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 1000000,
  col_types = colunas
)

```

```

computeStats <- function(stats) {

  stats %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma = sum(soma),
      total = sum(total),
      .groups = "drop"
    ) %>%
    mutate(
      media = soma / total
    )
}

flights <- computeStats(flights)

```

2. Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

- Identifique a coluna que é a chave na tabela `flights`;
- Identifique a coluna que é a chave na tabela `airports`;
- Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?
- Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
- Armazene a tabela resultante no objeto `flights`.

```

# A chave da tabela flights é DESTINATION_AIRPORT
# A chave da tabela airports é IATA_CODE
anti_join(
  flights,
  airports,

```

```
by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
```

```
# A tibble: 0 × 4
# i 4 variables: DESTINATION_AIRPORT <chr>, soma <dbl>, total <int>,
#   media <dbl>
```

```
flights <- left_join(
  flights,
  airports,
  by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
flights
```

```
# A tibble: 322 × 8
  DESTINATION_AIRPORT  soma total media CITY          STATE LATITUDE LONGITUDE
  <chr>              <dbl> <int> <dbl> <chr>          <chr>    <dbl>    <dbl>
1 ABE                12874  2220  5.80 Allentown      PA        40.7    -75.4
2 ABI                 9077  2224  4.08 Abilene        TX        32.4    -99.7
3 ABQ              106415 18953  5.61 Albuquerque   NM        35.0   -107.
4 ABR                -2227   657 -3.39 Aberdeen     SD        45.4    -98.4
5 ABY                 7501   864  8.68 Albany        GA        31.5    -84.2
6 ACK                 2393   487  4.91 Nantucket      MA        41.3    -70.1
7 ACT                16392  1541 10.6  Waco           TX        31.6    -97.2
8 ACV                12259  1267  9.68 Arcata/Eureka  CA        41.0   -124.
9 ACY                41744  3530 11.8  Atlantic City  NJ        39.5    -74.6
10 ADK                -284    89 -3.19 Adak           AK        51.9   -177.
# i 312 more rows
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports %>%
  group_by(STATE) %>%
  summarise(
    aeroportos = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(aeroportos))
```

```
# A tibble: 54 × 2
  STATE aeroportos
  <chr>         <int>
1 TX             24
2 CA             22
3 AK             19
4 FL             17
5 MI             15
6 NY             14
7 CO             10
8 MN              8
9 MT              8
```

10 NC

8

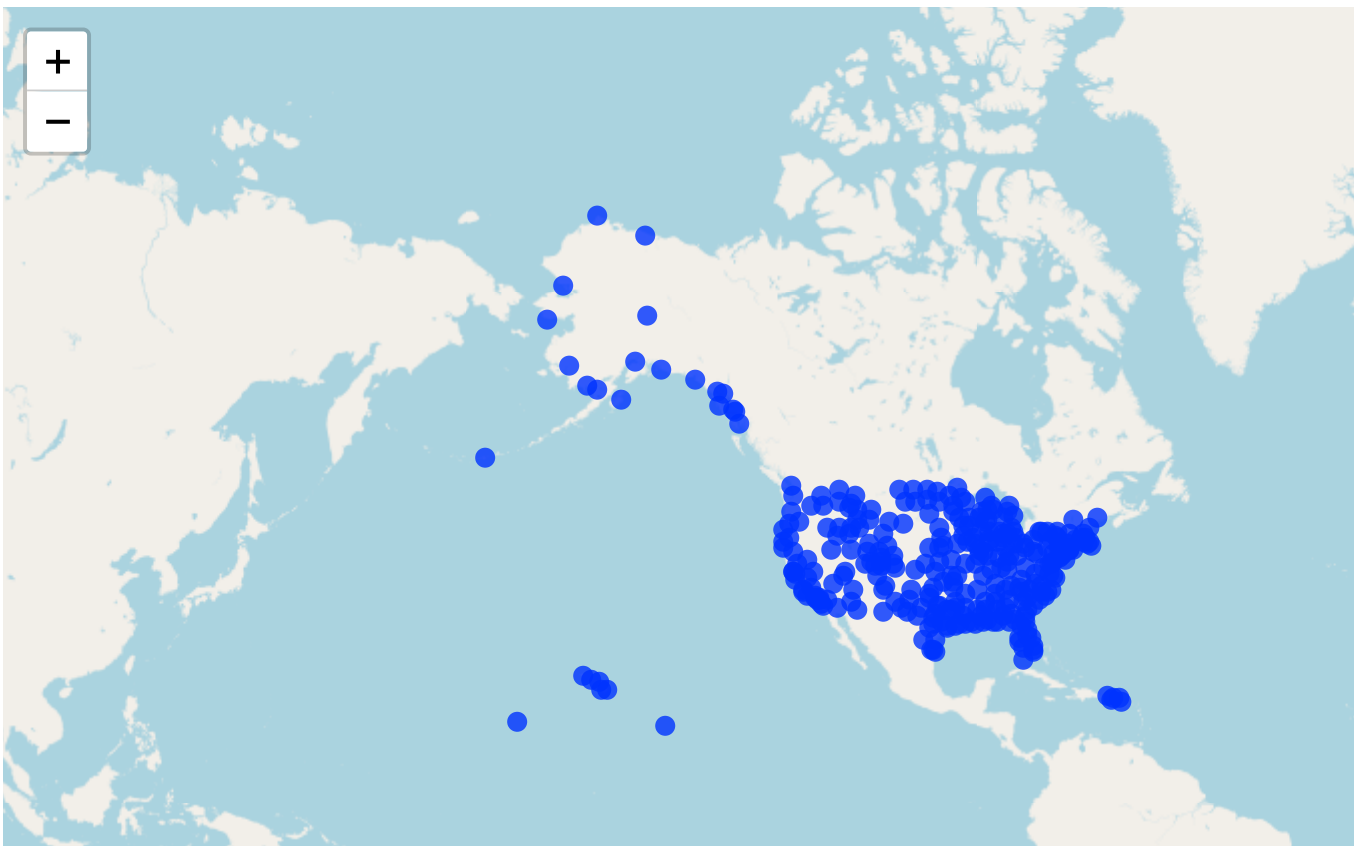
i 44 more rows

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

- Carregue o pacote `leaflet`;
- Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
- Armazene o grafico em b) numa variável chamada `this`;
- Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```
this <- leaflet(flights) %>%  
  addTiles() %>%  
  addCircleMarkers(  
    lng = ~LONGITUDE,  
    lat = ~LATITUDE,  
    popup = ~paste0(  
      "<b>Aeroporto:</b> ", DESTINATION_AIRPORT, "<br>",  
      "<b>Cidade/UF:</b> ", CITY, " - ", STATE, "<br>",  
      "<b>Atraso Médio:</b> ", round(media, 2), " min"  
    ),  
    radius = 5,  
    stroke = FALSE,  
    fillOpacity = 0.8  
  )
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

`this`

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 17:23:01 -03"
```